A LIVEBOOK NOTEBOOK · TALK.LIVEMD

Elixir for Gophers

Same problems, different solutions

Nikola Begedin · V7

All cells in this document ran on 3 November 2026, with Elixir 1.20.4, Erlang/OTP 28 and Go 1.27.2. The outputs come from that run. PIDs, times and node names are different in each run. The notebook ran outside Livebook, as the hidden node notebook@127.0.0.1. Thus no Livebook server node shows in the node lists.

Contents

2 Same problems, different solutions
3 Where we meet
4 Pattern matching
5 Function clauses and pipes
6 Protocols and behaviours
7 Processes vs goroutines
8 Processes vs goroutines, in code
9 Fan out, time out and cancel
10 Errors: same idea, different crash
11 Let it crash, with a supervisor
12 State without locks
13 Many nodes, one system
14 Look inside a running system
15 A web project: Gin and Phoenix
16 Talking to the database
17 Ecto in action
18 Go types and Elixir types
19 Type checking
20 Typespecs and Dialyzer
21 An Elixir app called go-backend
22 Boundaries in a big codebase
23 Compile times
24 When to use what
25 Use both: Ports
26 Takeaways
27 Questions
28 Backup: CPU speed
29 Backup: Deployment
30 Backup: Hiring and the ecosystem
31 Backup: Hot code upgrades

2 Same problems, different solutions

1. Language features
2. Runtime
3. Typical web project
4. Types
5. An example of a real codebase

---

3 Where we meet

- **Go:** *Go Developer Survey 2025* 74% build CLIs, 73% build API services, 46% build websites and 38% build cloud infrastructure tools.
- **Elixir:** *2025 Elixir community survey* 97% use Phoenix (web + API), 86% use LiveView.
- **Where we meet?** Web API.

Similarities

Go	Elixir
`go fmt`	`mix format`
`go test`	`mix test` (ExUnit)
`go doc`, pkg.go.dev	`h/1` in IEx, ExDoc and HexDocs
`go mod`, `go get`	`mix deps.get`, Hex
`if err ≠ nil`	`{:ok, value}` / `{:error, reason}`
structs and interfaces	structs, protocols and behaviours

Small languages. No classes. No inheritance.

---

4 Pattern matching

Elixir: `=` is a match. The left side is a pattern, and the match binds the variables in it.

```elixir
response = {:ok, %{name: "Ana", langs: ["Go", "Elixir"]}}

{:ok, %{name: name, langs: [first | _rest]}} = response
{name, first}
```

```
{"Ana", "Go"}
```

```elixir
{:ok, %{name: "Ana", langs: ["Go", second]}} = response
second
```

```
"Elixir"
```

```elixir
try do
  {:ok, %{name: "Annie"}} = response
rescue
  error in MatchError -> Exception.format(:error, error) |> Kino.Text.new(terminal: true)
end
```

```
** (MatchError) no match of right hand side value:

    {:ok, %{name: "Ana", langs: ["Go", "Elixir"]}}
```

Elixir: `case` runs the first branch whose pattern matches. Put the most specific patterns first.

```elixir
responses = [
  {:ok, %{status: 200, body: %{name: "Ana", langs: ["Go", "Elixir"]}}},
  {:ok, %{status: 200, body: %{name: "Bo", langs: []}}},
  {:ok, %{status: 404, body: %{error: "user not found"}}},
  {:error, :timeout}
]

for response <- responses do
  case response do
    {:ok, %{status: 200, body: %{name: name, langs: [first | _]}}} -> "#{name} writes #{first}"
    {:ok, %{status: 200, body: %{name: name, langs: []}}} -> "#{name} writes nothing yet"
    {:ok, %{status: status, body: %{error: message}}} -> "HTTP #{status}: #{message}"
    {:error, reason} -> "request failed: #{reason}"
  end
end
```

```
["Ana writes Go", "Bo writes nothing yet", "HTTP 404: user not found",
 "request failed: timeout"]
```

- If a value does not match the pattern, you get a `MatchError`. In a process, that is a crash.
- If no `case` clause matches, you get a `CaseClauseError`.
- `receive`, `with` and function clauses all use the same patterns.

---

5 Function clauses and pipes

Go: one function with a `switch` inside.

Elixir: one function with many clauses (function heads). The first clause whose pattern and guard (`when`) match is the one that runs.

```elixir
defmodule Status do
  def text(200), do: "OK"
  def text(404), do: "Not Found"
  def text(code) when code in 500..599, do: "Server Error"
  def text(_code), do: "Unknown"
end

for code <- [200, 404, 503, 302], do: {code, Status.text(code)}
```

```
[{200, "OK"}, {404, "Not Found"}, {503, "Server Error"}, {302, "Unknown"}]
```

- `def text(_code)` is the catch-all. It matches every code that the other clauses miss.
- Without it, an unknown code gives you a `FunctionClauseError`.

- `▷` is the pipe operator. It takes the value on its left and gives it to the next function as the first argument.
- So you can write nested calls as steps, and read them from top to bottom.

```elixir
title = "  elixir for gophers  "

# words = String.split(String.upcase(String.trim(title)))

words =
  title
  |> String.trim()
  |> String.upcase()
  |> String.split()

# can also be
# words = title |> String.trim() |> String.upcase() |> String.split()

{title, words}
```

```
{"  elixir for gophers  ", ["ELIXIR", "FOR", "GOPHERS"]}
```

---

6 Protocols and behaviours

Go: one interface type for both cases. A type satisfies it implicitly.

Elixir: two explicit tools.

- **Protocol:** a function with one implementation for each data type, like a Go interface for values. `to_string/1` uses the `String.Chars` protocol.
- **Behaviour:** a list of callbacks that a module must define, like a Go interface for modules. `GenServer` is a behaviour. A missing callback gives a compiler warning.

```elixir
defprotocol Describe do
  def describe(value)
end

defimpl Describe, for: Integer do
  def describe(n), do: "the number #{n}"
end

defimpl Describe, for: Map do
  def describe(map), do: "a map with #{map_size(map)} keys"
end

for value <- [42, %{a: 1, b: 2}], do: Describe.describe(value)
```

```
["the number 42", "a map with 2 keys"]
```

7 Processes vs goroutines

Neither is an OS thread or an OS process. Both runtimes run many of them on a few OS threads.

	Go: goroutines, CSP (Hoare, 1978)	BEAM: processes, actors (Hewitt, 1973)
The address	none: you talk to a channel	a PID, with a mailbox
Memory	one shared heap	own heap and own GC
Send	waits for a receiver	copies the message, does not wait
Receive	one channel, or `select`	own mailbox, by pattern
Backpressure	yes: a full channel blocks	no: a mailbox can grow
Failure	not in the model	links and monitors
Other machines	no	yes, with the same `send/2`

- **Myth:** “Go scheduling is cooperative.” Since Go 1.14, the runtime also stops a goroutine in a tight loop, after about 10 ms.
- **Myth:** “Elixir processes are insanely cheap.” Cheap compared with an OS thread, but both cost less than 3 KB. The difference is isolation.
- “Share memory by communicating” is a Go proverb, but a mutex is also normal Go.

Go has backpressure built in. The BEAM has addresses, crash signals and distribution built in.

8 Processes vs goroutines, in code

Go: start 100,000 goroutines that wait on a channel.

GO · GO/GOROUTINES/MAIN.GO

```go
const count = 100_000

func main() {
    var before, after runtime.MemStats
    runtime.GC()
    runtime.ReadMemStats(&before)

    stop := make(chan struct{})
    var wg sync.WaitGroup

    start := time.Now()
    for range count {
        wg.Go(func() { <-stop })
    }
    elapsed := time.Since(start)

    runtime.ReadMemStats(&after)
    used := inUse(after) - inUse(before)

    close(stop)
    wg.Wait()

    fmt.Printf("goroutines: %d\n", count)
    fmt.Printf("time: %d ms\n", elapsed.Milliseconds())
    fmt.Printf("bytes per goroutine: %d\n", used/count)
}

// inUse returns the bytes that goroutine stacks and the heap use.
func inUse(m runtime.MemStats) uint64 {
    return m.StackInuse + m.HeapInuse
}
```

```
$ go run ./go/goroutines

goroutines: 100000
time: 88 ms
bytes per goroutine: 2741

exit status 0
```

Elixir: start 100,000 processes that wait for a message.

ELIXIR

```elixir
defmodule Spawner do
  def start(count) do
    for _ <- 1..count do
      spawn(fn ->
        receive do
          :stop -> :ok
        end
      end)
    end
  end
end

before = :erlang.memory(:processes)
{ms, pids} = :timer.tc(fn -> Spawner.start(100_000) end, :millisecond)
used = :erlang.memory(:processes) - before
Enum.each(pids, &send(&1, :stop))

Kino.Text.new(
  """
  processes: 100000
  time: #{ms} ms
  bytes per process: #{div(used, 100_000)}
  """,
  terminal: true
)
```

```
processes: 100000
time: 62 ms
bytes per process: 2648
```

Go: a goroutine has no address. You talk to a channel.

GO · GO/CHANNELS/MAIN.GO

```go
func main() {
    ping := make(chan chan string)

    go func() {
        reply := <-ping
        reply <- "pong"
    }()

    reply := make(chan string)
    ping <- reply
    fmt.Println(<-reply)
}
```

```
$ go run ./go/channels

pong

exit status 0
```

Elixir: a process has an address. You send a message to its PID.

```elixir
pong =
  spawn(fn ->
    Process.set_label(:pong)

    receive do
      {:ping, from} -> send(from, :pong)
    end

    receive do
      :stop -> :ok
    end
  end)

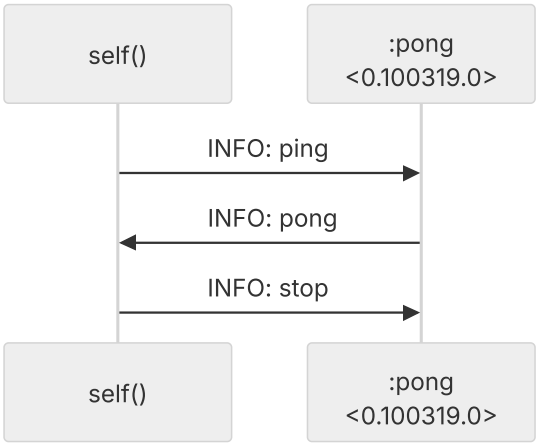
{:ok, diagram} =
  Kino.Process.seq_trace(fn ->
    send(pong, {:ping, self()})

    receive do
      :pong -> :ok
    end

    send(pong, :stop)

    :ok
  end)

diagram
```



Both runtimes start 100,000 tasks in milliseconds. A PID can get messages from any process, also on another node.

9 Fan out, time out and cancel

Check 100 links, 10 at a time. One link in ten hangs. Stop the wait after 500 ms.

Go: a semaphore channel limits the checks. `context.WithTimeout` stops the wait.

GO · GO/FANOUT/MAIN.GO

```go
func main() {
    start := time.Now()
    limit := make(chan struct{}, 10) // at most 10 checks at a time
    var wg sync.WaitGroup
    var ok, timeout atomic.Int64

    for link := range 100 {
        wg.Go(func() {
            limit <- struct{}{}
            defer func() { <-limit }()

            ctx, cancel := context.WithTimeout(context.Background(), 500*time.Millisecond)
            defer cancel()

            if err := check(ctx, link); err != nil {
                timeout.Add(1)
                return
            }
            ok.Add(1)
        })
    }
    wg.Wait()

    fmt.Printf("ok: %d, timeout: %d, time: %d ms\n",
        ok.Load(), timeout.Load(), time.Since(start).Milliseconds())
}

// check simulates one link check. It must watch ctx to stop early.
func check(ctx context.Context, link int) error {
    latency := time.Duration(rand.IntN(200)) * time.Millisecond
    if link%10 == 0 {
        latency = time.Hour // this link hangs
    }

    select {
    case <-time.After(latency):
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}
```

```
$ go run ./go/fanout

ok: 90, timeout: 10, time: 1728 ms

exit status 0
```

Elixir: `Task.async_stream/3` limits the tasks and stops a slow task.

ELIXIR

```elixir
check = fn link ->
  if rem(link, 10) == 0, do: Process.sleep(:infinity)
  :rand.uniform(200) |> Process.sleep()
end

{ms, results} =
  :timer.tc(
    fn ->
      0..99
      |> Task.async_stream(check, max_concurrency: 10, timeout: 500, on_timeout: :kill_task)
      |> Enum.frequencies_by(fn
        {:ok, _} -> :ok
        {:exit, :timeout} -> :timeout
      end)
    end,
    :millisecond
  )

Map.put(results, :ms, ms)
```

```
%{timeout: 10, ok: 90, ms: 1703}
```

- **Go:** you give `ctx` to every function, and `check` must watch `ctx.Done()`. A function that ignores `ctx` continues after the timeout or `cancel()`. Go 1.27 adds a goroutine leak profile to find these goroutines.
- **Elixir:** there is no `ctx` argument, and `check` knows nothing about timeouts. `Task.async_stream/3` kills a task that is too slow. `Task.shutdown/2` or `Process.exit/2` stops any process, wherever its code is.
- Links send the stop to the child processes too.

Elixir can stop any work from outside. Go makes cancellation visible in each signature.

---

10 Errors: same idea, different crash

Expected errors are values in both languages.

Go: each step checks and wraps the error.

GO · GO/SNIPPETS/ERRORS.GO

```go
// Expected errors are values. Each step checks and wraps the error.
func LoadUser(ctx context.Context, id string) (User, error) {
    row, err := db.QueryUser(ctx, id)
    if err != nil {
        return User{}, fmt.Errorf("query user %s: %w", id, err)
    }

    user, err := parseUser(row)
    if err != nil {
        return User{}, fmt.Errorf("parse user %s: %w", id, err)
    }

    return user, nil
}
```

Elixir: the same idea with `with`.

```elixir
def load_user(id) do
  with {:ok, row} <- Db.query_user(id),
       {:ok, user} <- parse_user(row) do
    {:ok, user}
  end
end
```

ELIXIR

Unexpected errors are different.

Go: a panic in one goroutine stops the whole program.

```go
func main() {
    for id := 1; id <= 3; id++ {
        go worker(id)
    }

    time.Sleep(time.Second)
    fmt.Println("main: still running") // never printed
}

func worker(id int) {
    time.Sleep(time.Duration(id) * 100 * time.Millisecond)
    if id == 2 {
        var counts map[string]int
        counts["boom"]++ // panic: assignment to entry in nil map
    }
    fmt.Println("worker", id, "done")
}
```

GO · GO/CRASH/MAIN.GO

```
$ go run ./go/crash

worker 1 done
panic: assignment to entry in nil map

goroutine 20 [running]:
main.worker(0x2)
    github.com/begedin/elixir-for-gophers/go/crash/main.go:23 +0x54
created by main.main in goroutine 1
    github.com/begedin/elixir-for-gophers/go/crash/main.go:12 +0x4c

exit status 2
```

Elixir: a crash stops one process. A monitor tells you why.

ELIXIR

```elixir
for id <- 1..3 do
  spawn_monitor(fn ->
    Process.sleep(id * 100)
    if id == 2, do: raise("boom")
    IO.puts("worker #{id} done")
  end)
end

reasons =
  for _ <- 1..3 do
    receive do
      {:DOWN, _ref, :process, _pid, :normal} -> :normal
      {:DOWN, _ref, :process, _pid, {error, _stack}} -> {:crashed, Exception.message(error)}
    end
  end

IO.puts("cell: still running")
reasons
```

STANDARD OUTPUT

```
worker 1 done

[error] Process #PID<0.100440.0> on node :"notebook@127.0.0.1" raised an exception
** (RuntimeError) boom
    (elixir 1.20.4) src/elixir.erl:382: :elixir.eval_external_handler/3
    (stdlib 7.2) erl_eval.erl:924: :erl_eval.do_apply/7
worker 3 done
cell: still running
```

```
[:normal, {:crashed, "boom"}, :normal]
```

Caveat: `net/http` recovers a panic in a handler. A panic in a goroutine that the handler starts still stops the server.

For unexpected errors, Elixir stops one process and Go stops the program.

---

11 Let it crash, with a supervisor

- A supervisor starts child processes and restarts them when they crash.
- Strategies:
  - `:one_for_one` restarts only the crashed child. Use it when the children are independent.
  - `:rest_for_one` restarts the crashed child and all children that started after it. Use it when later children depend on earlier ones.
  - `:one_for_all` restarts all children. Use it when the children work only as a group.
- Too many crashes in a short time: the supervisor stops too, and its own supervisor decides.
  - If the crashes continue, the stop moves all the way to the application supervisor, and the whole app stops. Many people who are new to Elixir do not expect this.
- **Go:** a `recover` loop around each goroutine, or Kubernetes restarts the whole program.

Elixir: a supervisor with three workers.

```elixir
defmodule Worker do
  use GenServer

  def start_link(id), do: GenServer.start_link(__MODULE__, id)

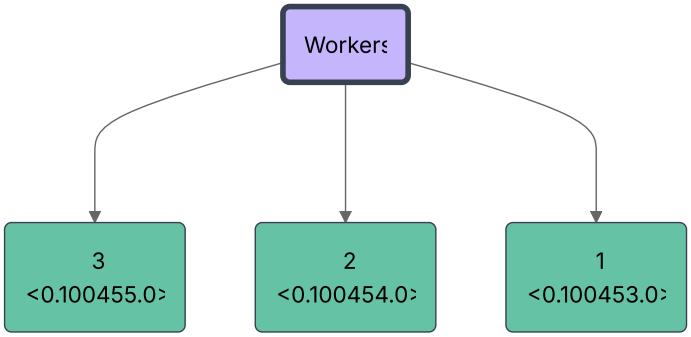
  @impl true
  def init(id), do: {:ok, %{id: id, started_at: Time.utc_now()}}
end

defmodule Workers do
  use Supervisor

  def start_link(_), do: Supervisor.start_link(__MODULE__, :ok, name: __MODULE__)

  @impl true
  def init(:ok) do
    children = for id <- 1..3, do: Supervisor.child_spec({Worker, id}, id: id)
    Supervisor.init(children, strategy: :one_for_one)
  end
end

Kino.start_child(Workers)
Kino.Process.sup_tree(Workers)
```

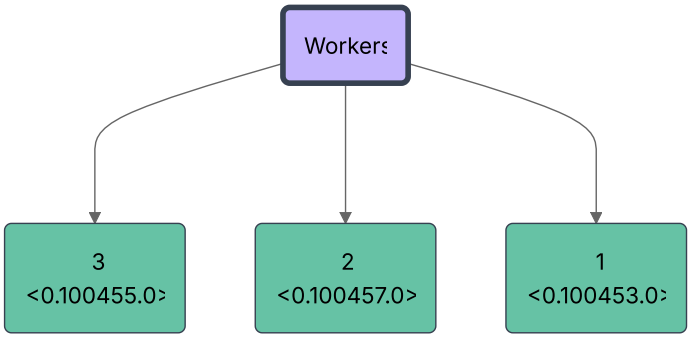


Kill worker 2. The supervisor starts a new worker 2 with a new PID.

```elixir
{2, pid, :worker, _} = Workers |> Supervisor.which_children() |> List.keyfind(2, 0)
Process.exit(pid, :kill)
Process.sleep(100)

Kino.Layout.grid([
  Kino.Text.new("killed #{inspect(pid)}"),
  Kino.Process.sup_tree(Workers)
])
```

```
killed #PID<0.100454.0>
```



Elixir restarts small parts of a program. Go usually restarts the whole program, with help from the platform.

---

12 State without locks

100 callers increment a counter at the same time, 1,000 times each.

Go: shared memory. One counter has a mutex, the other counter has no lock.

GO · GO/COUNTER/MAIN.GO

```go
const (
    callers    = 100
    increments = 1_000
)

// Counter is safe for concurrent use. The mutex protects n.
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) Increment() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}

func main() {
    var safe Counter
    run(safe.Increment)

    unsafe := 0
    run(func() { unsafe++ }) // no lock: a data race

    fmt.Println("expected:     ", callers*increments)
    fmt.Println("with mutex:   ", safe.n)
    fmt.Println("without mutex:", unsafe)
}

// run calls increment from many goroutines at the same time.
func run(increment func()) {
    var wg sync.WaitGroup
    for range callers {
        wg.Go(func() {
            for range increments {
                increment()
            }
        })
    }
    wg.Wait()
}
```

```
$ go run ./go/counter

expected:      100000
with mutex:    100000
without mutex: 37501

exit status 0
```

Go: the same program, built with `-race`. The race detector finds the missing lock.

```
$ go run -race ./go/counter

==================
WARNING: DATA RACE
Read at 0x00c0001b4008 by goroutine 109:
  main.main.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x2c
  main.run.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:48 +0x3c

Previous write at 0x00c0001b4008 by goroutine 107:
  main.main.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x3c
  main.run.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:48 +0x3c

Goroutine 109 (running) created at:
  sync.(*WaitGroup).Go()
      sync/waitgroup.go:238 +0x6c
  main.main()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x11c

Goroutine 107 (finished) created at:
  sync.(*WaitGroup).Go()
      sync/waitgroup.go:238 +0x6c
  main.main()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x11c
==================
==================
WARNING: DATA RACE
Write at 0x00c0001b4008 by goroutine 115:
  main.main.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x3c
  main.run.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:48 +0x3c

Previous write at 0x00c0001b4008 by goroutine 109:
  main.main.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x3c
  main.run.func1()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:48 +0x3c

Goroutine 115 (running) created at:
  sync.(*WaitGroup).Go()
      sync/waitgroup.go:238 +0x6c
  main.main()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x11c

Goroutine 109 (running) created at:
  sync.(*WaitGroup).Go()
      sync/waitgroup.go:238 +0x6c
  main.main()
      github.com/begedin/elixir-for-gophers/go/counter/main.go:35 +0x11c
==================
expected:      100000
with mutex:    100000
without mutex: 31654
Found 2 data race(s)

exit status 66
```

Elixir: the state is inside one process.

```elixir
defmodule Counter do
  use GenServer

  def start_link(_), do: GenServer.start_link(__MODULE__, 0, name: __MODULE__)
  def increment, do: GenServer.call(__MODULE__, :increment)
  def value, do: GenServer.call(__MODULE__, :value)

  @impl true
  def init(n), do: {:ok, n}

  @impl true
  def handle_call(:increment, _from, n), do: {:reply, :ok, n + 1}
  def handle_call(:value, _from, n), do: {:reply, n, n}
end

Kino.start_child(Counter)

1..100
|> Task.async_stream(fn _ -> for _ <- 1..1_000, do: Counter.increment() end, max_concurrency: 100)
|> Stream.run()

Counter.value()
```

```
100000
```

- **Go:** without the mutex, increments get lost. `-race` finds a race only when it occurs during the run.
- **Elixir:** the process handles one message at a time, so it needs no lock.
- For many concurrent readers, use ETS: an in-memory table with concurrent reads.

A Go mutex is faster for hot shared memory. Elixir has no data races, but logic races are still possible.

---

13 Many nodes, one system

- Distributed Erlang connects BEAM nodes. `send/2` works the same for a PID on another node.
- Phoenix.PubSub, Channels and LiveView use this to send events to every node in a cluster.
- **Go:** gRPC, NATS or Redis between services, plus service discovery.

Start two more nodes on this laptop. By default, visible nodes connect in a full mesh: when one node connects to `alpha` and `beta`, they also connect to each other. `-connect_all false` turns that off, so the cells below can show each connection.

ELIXIR

```elixir
start_node = fn prefix ->
  {:ok, peer, node} =
    :peer.start_link(%{
      # A unique name, so the cell can run again while old nodes still run
      name: :peer.random_name(prefix),
      host: ~c"127.0.0.1",
      longnames: true,
      connection: :standard_io,
      exec: :code.root_dir() |> Path.join("bin/erl") |> String.to_charlist(),
      args: [
        ~c"-setcookie",
        Node.get_cookie() |> Atom.to_charlist(),
        ~c"-connect_all",
        ~c"false"
      ]
    })

  :ok = :peer.call(peer, :code, :add_paths, [:code.get_path()])
  true = Node.connect(node)
  {peer, node}
end

{alpha_peer, alpha} = start_node.(:alpha)
{beta_peer, beta} = start_node.(:beta)
[alpha, beta]
```

```
[:"alpha-20034-57404@127.0.0.1", :"beta-20162-57404@127.0.0.1"]
```

Start a process on `beta` and send a message to it.

ELIXIR

```elixir
pid =
  Node.spawn(beta, fn ->
    receive do
      {:hello, from} -> send(from, {:hello_back, node()})
    end
  end)

send(pid, {:hello, self()})

receive do
  reply -> reply
after
  1_000 -> :timeout
end
```

```
{:hello_back, :"beta-20162-57404@127.0.0.1"}
```

Which nodes do `alpha` and this notebook see? `alpha` sees only the notebook. Livebook runs this notebook as a hidden node, and `Node.list/0` shows only visible nodes, so the cell uses `Node.list(:connected)`. The `livebook_…` node in the notebook's list is the Livebook server.

ELIXIR

```elixir
seen = fn ->
  [Node.self() | Node.list(:connected)]
  |> Map.new(&{&1, :erpc.call(&1, Node, :list, [:connected])})
end

seen.()
```

```
%{
  "notebook@127.0.0.1": [:"alpha-20034-57404@127.0.0.1", :"beta-20162-57404@127.0.0.1"],
  "alpha-20034-57404@127.0.0.1": [:"notebook@127.0.0.1"],
  "beta-20162-57404@127.0.0.1": [:"notebook@127.0.0.1"]
}
```

Connect `alpha` to `beta`, and list again. Now `alpha` sees `beta` too.

ELIXIR

```elixir
true = :erpc.call(alpha, Node, :connect, [beta])
seen.()
```

```
%{
  "notebook@127.0.0.1": [:"alpha-20034-57404@127.0.0.1", :"beta-20162-57404@127.0.0.1"],
  "alpha-20034-57404@127.0.0.1": [:"notebook@127.0.0.1", :"beta-20162-57404@127.0.0.1"],
  "beta-20162-57404@127.0.0.1": [:"notebook@127.0.0.1", :"alpha-20034-57404@127.0.0.1"]
}
```

Stop `beta`.

ELIXIR

```elixir
:peer.stop(beta_peer)
seen.()
```

```
%{
  "notebook@127.0.0.1": [:"alpha-20034-57404@127.0.0.1"],
  "alpha-20034-57404@127.0.0.1": [:"notebook@127.0.0.1"]
}
```

List again. Nobody sees `beta` now. Then stop `alpha` too.

ELIXIR

```elixir
:peer.stop(alpha_peer)
seen.()
```

```
%{"notebook@127.0.0.1": []}
```

Elixir has clustering built in, for one trusted system. Go with gRPC is better across languages and trust boundaries.

14 Look inside a running system

Go has good tools for CPU and memory profiling. Elixir lets you look inside a live system and change it.

- In production, `bin/my_app remote` opens an IEx shell inside the running release.
- Phoenix LiveDashboard and Oban Web show the same data in a browser.
- You can read the state of any process and trace its messages, with no restart.
- **Go:** pprof, `runtime/trace` with the flight recorder (Go 1.25), the goroutine leak profile (Go 1.27).

Read the state of worker 1 from the supervisor slide.

```elixir
{1, pid, _, _} = Workers |> Supervisor.which_children() |> List.keyfind(1, 0)

%{
  state: :sys.get_state(pid),
  info: Process.info(pid, [:memory, :message_queue_len, :reductions]),
  processes_on_this_node: Process.list() |> length()
}
```

```
%{
  info: [memory: 2888, message_queue_len: 0, reductions: 566],
  state: %{id: 1, started_at: ~T[10:58:14.926512]},
  processes_on_this_node: 190
}
```

15 A web project: Gin and Phoenix

Go lets each team select its layout. Phoenix gives all projects the same layout, so a new developer knows where to look.

- **Go:** Gin is the most used Go web framework (48%). Many teams use only the standard library `net/http` (32%). *JetBrains, Go ecosystem surveys 2025*
- **Elixir:** `mix phx.new my_app` generates a Phoenix project.

	Go with Gin (a common layout)	Phoenix 1.8
Start	`cmd/api/main.go`	`lib/my_app/application.ex`, a supervisor
Routes	`r.GET("/users/:id", h.GetUser)`	`get "/users/:id", UserController, :show`
Handlers	`internal/handler/`	`lib/my_app_web/controllers/`
Business logic	`internal/service/`	`lib/my_app/accounts.ex`, a context
Database	`internal/store/` with sqlc	`lib/my_app/repo.ex` and Ecto schemas

- **Go:** the Go team documents only `cmd/` and `internal/`. Each team selects the rest.
- **Elixir:** `my_app` holds the business logic, and `my_app_web` holds the HTTP code. The generators add code in the same places.

16 Talking to the database

Go:

- `database/sql` has a connection pool built in. pgx is the usual Postgres driver.
- sqlc generates typed Go code from SQL. GORM is the popular ORM. sqlx scans rows into structs.
- Migrations: goose, golang-migrate or Atlas.

GO · GO/SNIPPETS/SQLC.GO

```go
// sqlc reads this SQL and generates the Go code below:
//
//  -- name: ListActiveUsers :many
//  SELECT id, name, email FROM users
//  WHERE active AND inserted_at > $1
//  ORDER BY name;

func (q *Queries) ListActiveUsers(ctx context.Context, since time.Time) ([]User, error) {
    rows, err := q.db.Query(ctx, listActiveUsers, since)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var users []User
    for rows.Next() {
        var u User
        if err := rows.Scan(&u.ID, &u.Name, &u.Email); err != nil {
            return nil, err
        }
        users = append(users, u)
    }
    return users, rows.Err()
}
```

Elixir:

- Ecto has four parts: Repo, Schema, Changeset and Query. Postgrex is the driver. DBConnection is the pool.
- Ecto is not an ORM. A schema is a struct, not an object. There is no lazy loading: you preload associations.
- Migrations are part of Ecto (`mix ecto.migrate`).

ELIXIR

```elixir
# pipe form
def list_active_users(since) do
  User
  |> where([u], u.active and u.inserted_at > ^since)
  |> order_by(:name)
  |> select([:id, :name, :email])
  |> Repo.all()
end

# options/keyword form
def list_active_users(since) do
  query =
    from u in User,
      where: u.active and u.inserted_at > ^since,
      order_by: u.name,
      select: [:id, :name, :email]

  Repo.all(query)
end
```

sqlc checks your SQL when you generate the code. Ecto queries compose, and validation errors are data.

---

17 Ecto in action

A schema, a changeset and a SQLite database in a temporary file.

ELIXIR

```elixir
defmodule Demo.Repo do
  use Ecto.Repo, otp_app: :demo, adapter: Ecto.Adapters.SQLite3
end

defmodule Demo.User do
  use Ecto.Schema
  import Ecto.Changeset

  schema "users" do
    field :name, :string
    field :email, :string
    field :active, :boolean, default: true
    timestamps()
  end

  def changeset(%__MODULE__{} = user, attrs) do
    user
    |> cast(attrs, [:name, :email, :active])
    |> validate_required([:name, :email])
    |> validate_format(:email, ~r/@/)

    # %Ecto.Changeset{valid?: true, errors: [], data: %User{}, changes: %{...}}

  end
end

database = System.tmp_dir!() |> Path.join("elixir_for_gophers.db")
File.rm(database)
Kino.start_child({Demo.Repo, database: database, pool_size: 1, log: false})

Demo.Repo.query!("""
CREATE TABLE users (
  id INTEGER PRIMARY KEY,
  name TEXT, email TEXT, active BOOLEAN,
  inserted_at TEXT, updated_at TEXT
)
""")
```

```
%Exqlite.Result{command: :execute, columns: [], rows: [], num_rows: 0}
```

A transaction is all or nothing. The second insert fails, so the first insert does not stay. Validation errors are data, not exceptions.

ELIXIR

```elixir
alias Demo.{Repo, User}

eve = %User{} |> User.changeset(%{name: "Eve", email: "eve@example.com"})
mal = %User{} |> User.changeset(%{name: "Mal", email: "no email"})

IO.inspect({eve, mal})

{:error, failed_step, changeset, _changes_so_far} =
  Ecto.Multi.new()
  |> Ecto.Multi.insert(:eve, eve)
  |> Ecto.Multi.insert(:mal, mal)
  |> Repo.transact()

%{failed_step: failed_step, errors: changeset.errors, users_in_db: Repo.aggregate(User, :count)}
```

STANDARD OUTPUT

```
{#Ecto.Changeset<
   action: nil,
   changes: %{name: "Eve", email: "eve@example.com"},
   errors: [],
   data: #Demo.User<>,
   valid?: true,
   ...
 >,
 #Ecto.Changeset<
   action: nil,
   changes: %{name: "Mal", email: "no email"},
   errors: [email: {"has invalid format", [validation: :format]}],
   data: #Demo.User<>,
   valid?: false,
   ...
 >}
```

```
%{
  errors: [email: {"has invalid format", [validation: :format]}],
  failed_step: :mal,
  users_in_db: 0
}
```

- The SQL Sandbox gives **each test its own transaction**, so tests that use Postgres can run with `async: true`.

ELIXIR

```elixir
Repo.insert(eve)
Repo.all(User)
```

```
[
  %Demo.User{
    __meta__: #Ecto.Schema.Metadata<:loaded, "users">,
    id: 1,
    name: "Eve",
    email: "eve@example.com",
    active: true,
    inserted_at: ~N[2026-11-03 10:58:17],
    updated_at: ~N[2026-11-03 10:58:17]
  }
]
```

18 Go types and Elixir types

Go	Elixir
`int64` : wraps on overflow	integer with no size limit: `2 ** 100` is exact
`float64` , `float32`	float: always 64-bit. `7 / 2` is `3.5` , `div(7, 2)` is `3`
`string` , `[]byte`	string (UTF-8 bytes), binary (any bytes)
`[]T`	list: a linked list, so index access is slow
multiple return values: `(value, err)`	tuple: a fixed-size value, for example `{:ok, value}`
`map[K]V`	map, with keys and values of any type
`struct`	struct: a map with fixed keys, and no field types
`const` with `iota`	atoms: `:active` , `:banned`
pointers	none: all data is immutable

Go: a variable has a type. **Elixir:** a value has a type, and a variable can hold any value.

19 Type checking

- **Go:** static types and generics (generic methods since Go 1.27). The compiler rejects a program with a type error.
- **Elixir:** gradual types. The compiler infers types from patterns and guards, with no annotations.

Elixir 1.20: the compiler finds the type error in this module.

ELIXIR

```elixir
defmodule Shapes do
  def area({:square, side}), do: side * side
  def area({:circle, radius}), do: 3.14 * radius * radius

  def describe(shape), do: "Area: " <> area(shape)
end
```

WARNING · TALK.LIVEMD:834

```
incompatible types in binary construction:

    <<..., area(shape)::binary>>

got type:

    dynamic(float() or integer())

but expected type:

    binary()

where "shape" was given the type:

    # type: dynamic({:circle or :square, float() or integer()})
    # from: talk.livemd:834
    area(shape)
```

20 Typespecs and Dialyzer

- **Go:** the types are part of the code, and the compiler checks them. Go needs no second tool.
- **Elixir:** `@type` and `@spec` describe the types of a function. They are optional.

```elixir
@type shape :: {:square, number()} | {:circle, number()}

@spec area(shape()) :: number()
def area({:square, side}), do: side * side
```

- Dialyzer (`mix dialyzer` with the dialyxir package) reads the typespecs. It reports only code that can never work, so it misses many bugs.
- On a large project, Dialyzer is slow, and its messages are hard to read.
- The Elixir 1.20 compiler does not read `@spec`. It infers its own types.

Go gives stronger guarantees today. Elixir finds more bugs with each release, with no change to your code.

21 An Elixir app called go-backend

One Phoenix app, not an umbrella. The numbers are from `main` in October 2026.

Contexts	Modules	Lines of code in `lib/`	Test files
124	5,018	about 557,000	1,896

- 70% of the test files run with `async: true`.
- About 150 Oban workers, 11 Ecto repos, about 1,200 migrations.
- Other languages in the BEAM: a Rust NIF for text processing, and Python through Snex.
- One release with four node roles (`:api`, `:core`, `:worker`, `:reporting`) in one cluster.
- Only 16 hand-written GenServers. Phoenix, Oban and Broadway own most processes.

22 Boundaries in a big codebase

- Elixir has no `internal/` packages. Any module can call any other module.
- Boundary adds this check to the compiler. Each boundary lists its `deps` and `exports`. A call outside these rules is a compiler warning.
- go-backend has 149 boundaries. `dirty_xrefs` lists the known exceptions in old code.
- **Go:** `internal/` and the rule against import cycles are part of the toolchain.

```elixir
defmodule AgiDb.Organizations do
  use Boundary,
    deps: [AgiDb.Billing, AgiDb.Events, AgiDb.IAM, AgiDb.Repo, AgiDb.Workspaces, ...],
    exports: [Organization, Commands.CreateOrganization, Events.OrganizationWorkspaceAdded, ...]

  use AgiDb.Knigge,
    otp_app: :agidb,
    config_key: :organizations,
    default: AgiDb.Organizations.Internal.OrganizationsImpl

  @callback create_organization(Commands.CreateOrganization.t()) :: ...
end
```

- Each context is a facade: `@callback`s plus Knigge. Knigge sends each call to the implementation from the config.
- Tests replace the implementation with a Hammox mock (Mox with typespec checks). go-backend has 119 facades.
- Knigge's last release is from 2021. go-backend wraps it to remove a compile-time dependency.

---

23 Compile times

- **Go:** a fast compiler and the build cache are design goals. Unused imports are errors.
- **Elixir:** modules depend on each other at compile time, through exports, or at runtime. A change recompiles the compile-time dependents.
- Macros and `use` add compile-time dependencies. One small change can recompile half of a big app.
- Tools: `mix xref graph --label compile-connected`, `mix xref trace`, and `--fail-above` for CI.
- Elixir 1.19 compiles large projects up to 4 times faster. Elixir 1.20 is faster again on many cores.

go-backend checks this in CI.

```shell
# No more than 10 compile-connected cycles
mix xref graph --format cycles --label compile-connected --fail-above 10

# definitions.ex files must not depend on other project modules
mix xref graph --source lib/agidb/.../definitions.ex --fail-above 0
```

---

24 When to use what

Go is the better choice for	Elixir is the better choice for
CLIs and developer tools: one static binary, fast start	Many long-lived connections: chat, presence, WebSockets
Infrastructure: Kubernetes operators, proxies, agents	Real-time features and steady latency under load
CPU-heavy work and tight memory limits	Systems that must stay up when parts fail
Many small services on a platform	One product codebase with few moving parts

25 Use both: Ports

A Port starts an OS program and connects to its stdin and stdout. Go does the CPU work. Elixir owns the OS process.

GO · GO/PORT/MAIN.GO

```go
func main() {
    if len(os.Args) > 1 {
        for _, arg := range os.Args[1:] {
            printCount(arg)
        }
        return
    }

    scanner := bufio.NewScanner(os.Stdin)
    for scanner.Scan() {
        printCount(scanner.Text())
    }
}

// printCount reads text as a number n and prints the count of primes below n.
func printCount(text string) {
    n, err := strconv.Atoi(strings.TrimSpace(text))
    if err != nil {
        fmt.Println("error:", err)
        return
    }
    fmt.Println(countPrimes(n))
}

// countPrimes returns the number of primes below n.
// It uses the sieve of Eratosthenes.
func countPrimes(n int) int {
    if n < 3 {
        return 0
    }
    composite := make([]bool, n)
    count := 0
    for i := 2; i < n; i++ {
        if composite[i] {
            continue
        }
        count++
        for j := i * i; j < n; j += i {
            composite[j] = true
        }
    }
    return count
}
```

Run it once with `System.cmd`. The program starts, counts and stops.

ELIXIR

```elixir
bin = Slides.go_build("port")
{output, 0} = System.cmd(bin, ["10", "1000", "10000000"])
String.split(output)
```

```
["4", "168", "664579"]
```

```elixir
bin
```

```
"/var/folders/g2/p6bbc7sj2hq46p01dt0rw6w00000gn/T/elixir_for_gophers_port"
```

A Port keeps the program running. Send a number, receive the count of primes below it.

```elixir
port = Port.open({:spawn_executable, bin}, [:binary, :exit_status, line: 1024])

count_primes = fn n ->
  Port.command(port, "#{n}\n")

  receive do
    {^port, {:data, {:eol, line}}} -> String.to_integer(line)
  after
    5_000 -> raise "no reply from Go"
  end
end

%{
  os_pid: Port.info(port, :os_pid),
  counts: for(n <- [10, 1_000, 10_000_000], do: {n, count_primes.(n)})
}
```

```
%{os_pid: {:os_pid, 57583}, counts: [{10, 4}, {1000, 168}, {10000000, 664579}]}
```

Close the Port. Go reads the end of stdin and stops.

```elixir
Port.close(port)
```

```
true
```

---

26 Takeaways

- Same problems: concurrency, failures, state, data.
- Go puts the solutions in your code and in the platform. Elixir puts them in the runtime.
- Go's types are stronger today. Pattern matching and pipes change how Elixir code reads.
- Choose Go for tools, infrastructure and CPU-heavy services.
- Choose Elixir for connected, real-time products that must stay up when parts fail.
- You can use both, through a Port, a NIF or a network call.

Slides and code: github.com/begedin/elixir-for-gophers

27 Questions

28 Backup: CPU speed

The same sieve of Eratosthenes, for the numbers below 10,000,000, on an Apple M3 Pro.

	Time
Go	17 ms
Elixir: a compiled module, the JIT, an `:atomics` array	about 440 ms

- The BEAM is slow for tight loops over numbers, also with the JIT (since OTP 24).
- For CPU-heavy work, use a Port (like the Go program in this talk), a NIF in Rust with Rustler, or Nx for numerical work.
- Most web backends wait for the database and the network, not for the CPU.

---

29 Backup: Deployment

	Go	Elixir
Artifact	one static binary	a release (`mix release`): your code and the runtime
Minimal size	5.2 MB, with `net/http`, stripped	16 MB
Start	less than 10 ms	about 0.25 s for the VM, more for a Phoenix app
Cross-compile	yes: `GOOS` and `GOARCH`	no: build on the target OS and CPU, usually in Docker
Cluster	not necessary	DNSCluster (in Phoenix 1.8) or libcluster finds the nodes

- Measured on this laptop, with a new `mix new --sup` project and a small Go program.
- `mix phx.gen.release --docker` generates a release and a Dockerfile for a Phoenix app.
- Both run well on Kubernetes.

30 Backup: Hiring and the ecosystem

	Go	Elixir
Used in the last year	16.4% of the respondents	2.7% of the respondents
Admired	56.5%: 8th of 42 languages	65.9%: 3rd, after Rust and Gleam
Packages	no central registry: each Git repository is a module	Hex: about 26,700 packages
Cloud SDKs	official SDKs from AWS and Kubernetes	community packages (ExAws, k8s)
Hiring	a large pool of developers	a small pool: hire from other languages and teach Elixir
Static types	in the language	gradual types in Elixir, or Gleam: static types on the BEAM

- Source for the first two rows: Stack Overflow Developer Survey 2025.
- “Admired”: the users of a language who want to use it again next year.

31 Backup: Hot code upgrades

- The BEAM can load a new version of a module into a running system. The old and the new version can run at the same time.
- A full hot upgrade of a release needs appup and relup files. Few teams use it in production.
- Most teams deploy new nodes with a rolling deploy, like a Go service.
- In development, Phoenix code reloading and IEx `recompile/0` use the same feature every day.

A Appendix: the setup cell

The first cell of the notebook installs the dependencies and defines the `Slides` helpers. `Slides.go_source/2` shows the Go listings and `Slides.go_run/2` builds and runs the Go programs.

ELIXIR

```elixir
Mix.install([
  {:kino, "~> 0.19"},
  {:makeup_syntect, "~> 0.1"},
  {:ecto_sqlite3, "~> 0.25"}
])

defmodule Slides do
  @root __DIR__

  @doc """
  Shows a Go source file with syntax highlighting. The path is relative
  to the repository root.

  Livebook does not highlight Go code blocks, so this function
  highlights the source with Makeup and shows it as HTML.

  ## Options

    * `:from` - show the source from the first line that starts with
      this text. Use it to hide the package clause and the imports.
  """
  def go_source(path, opts \\ []) do
    {lexer, lexer_opts} = Makeup.Registry.fetch_lexer_by_name!("go")

    source =
      @root
      |> Path.join(path)
      |> File.read!()
      |> drop_header(opts[:from])

    html = Makeup.highlight(source, lexer: lexer, lexer_options: lexer_opts)
    css = Makeup.Styles.HTML.StyleMap.one_dark_style() |> Makeup.stylesheet()

    Kino.HTML.new("""
    <style>
      #{css}
      body { margin: 0; }
      .highlight {
        margin: 0;
        padding: 16px;
        border-radius: 8px;
        font-family: "JetBrains Mono", monospace;
        font-size: 14px;
        line-height: 1.5;
        tab-size: 4;
        overflow-x: auto;
      }
    </style>
    #{html}
    """)
  end

  @doc """
  Builds the Go program in `go/<name>` and runs it once.
  Shows the output and the exit status.

  ## Options
```

```
    * `:race` – build the program with the race detector (`-race`)
  """
  def go_run(name, opts \\ []) do
    {output, status} = name |> go_build(opts) |> System.cmd([], stderr_to_stdout: true)
    Kino.Text.new(output <> "\nexit status #{status}", terminal: true)
  end

  @doc """
  Builds the Go program in `go/<name>` and returns the path of the binary.
  Takes the same options as `go_run/2`.
  """
  def go_build(name, opts \\ []) do
    go = System.find_executable("go") || raise "go is not in PATH"
    race = Keyword.get(opts, :race, false)
    bin = System.tmp_dir!() |> Path.join("elixir_for_gophers_#{name}#{if race, do: "_race"}")
    flags = if race, do: ["-race"], else: []
    args = ["build", "-trimpath"] ++ flags ++ ["-o", bin, "./go/#{name}"]

    case System.cmd(go, args, cd: @root, stderr_to_stdout: true) do
      {_output, 0} -> bin
      {output, _status} -> raise "go build failed:\n#{output}"
    end
  end

  # Without `from`, removes only the `//go:build` line of a snippet.
  defp drop_header(source, nil), do: String.replace(source, ~r/\A\/\/go:build[^\n]*\n+/, "")

  defp drop_header(source, from) do
    [_header, rest] = String.split(source, "\n" <> from, parts: 2)
    from <> rest
  end
end
```